Calculating Peak GPU Memory Usage from Code Constants: GPT-2 with GPipe Pipeline Parallelism

1 Introduction

This document describes how to calculate the theoretical peak GPU memory usage for the GPT-2 pipeline-parallel training script `gpt2_gpipe_memory_profile.py` using only the constants defined in the source code. We derive closed-form formulas for each memory component and show how they compose to predict the observed peak allocation of $\approx 625.77\,\text{MB}$ on GPU 0 (from the example trace `peak_alloc_events_GPU0_for example.json`, where `max_allocated_mem` = 656 165 888 bytes).

2 Code Constants

All quantities below are drawn directly from the source file.

2.1 Training Constants

Symbol	Code Variable	Value
B	`BATCH_SIZE`	8
G	`NUM_GPUS`	`torch.cuda.device_count()`
C	`MICRO_BATCHES`	G
S	`SEQ_LEN`	256
T	`NUM_STEPS`	10

2.2 GPT-2 Model Constants

Symbol	Code Variable / Config Key	Value
V	`vocab_size`	50 257
P	`n_positions`	1 024
d	`n_embd`	768
L	`n_layer`	12
H	`n_head`	12
d_h	d/H	64
d_{ff}	`n_inner`	3 072

2.3 Derived Constants

- **Bytes per parameter**: $b = 4$ (all parameters are `float32`).
- **Micro-batch size**: $B_\mu = B/C = B/G$.
- **Total sequential modules**: $M = 1+L+1 = 14$ (1 `EmbeddingBlock` + 12 `TransformerBlock` + 1 `LMHead`).

- **Balance per GPU**: the code distributes M modules by

$$\text{balance}[i] = \left\lfloor \frac{M}{G} \right\rfloor + \mathbf{1}[i < M \bmod G], \quad i = 0, \ldots, G-1.$$

3 Memory Components

GPU memory during training is consumed by five main categories:

$$\boxed{\mathcal{M}_{\text{peak}}^{(i)} = \mathcal{M}_{\text{param}}^{(i)} + \mathcal{M}_{\text{grad}}^{(i)} + \mathcal{M}_{\text{opt}}^{(i)} + \mathcal{M}_{\text{act}}^{(i)} + \mathcal{M}_{\text{misc}}^{(i)}} \tag{1}$$

where i denotes the GPU index.

3.1 Model Parameters $\mathcal{M}_{\text{param}}$

3.1.1 EmbeddingBlock (Module 0)

$$\theta_{\texttt{wte}} = V \times d = 50\,257 \times 768 = 38\,597\,376 \tag{2}$$
$$\theta_{\texttt{wpe}} = P \times d = 1\,024 \times 768 = 786\,432 \tag{3}$$
$$\theta_{\text{emb}} = \theta_{\texttt{wte}} + \theta_{\texttt{wpe}} = 39\,383\,808 \tag{4}$$

3.1.2 Single TransformerBlock (Modules 1–12)

Each transformer block contains:

Sub-module	**Formula**	**Parameters**
`ln_1` (LayerNorm)	$2d$ (weight + bias)	1 536
`c_attn` (Linear)	$d \times 3d + 3d$	1 771 776
`c_proj` (Linear, Attn)	$d \times d + d$	590 592
`ln_2` (LayerNorm)	$2d$	1 536
`c_fc` (Linear, MLP)	$d \times d_{\text{ff}} + d_{\text{ff}}$	2 362 368
`c_proj` (Linear, MLP)	$d_{\text{ff}} \times d + d$	2 360 064
Total per block θ_{block}	$2(2d) + d(3d) + 3d + d^2 + d + d \cdot d_{\text{ff}} + d_{\text{ff}} + d_{\text{ff}} \cdot d + d$	**7 087 872**

In closed form:

$$\theta_{\text{block}} = 4d + 3d^2 + 3d + d^2 + d + d \cdot d_{\text{ff}} + d_{\text{ff}} + d_{\text{ff}} \cdot d + d = 4d^2 + 2d \cdot d_{\text{ff}} + d_{\text{ff}} + 9d \tag{5}$$

3.1.3 LMHead (Module 13)

$$\theta_{\texttt{ln_f}} = 2d = 1\,536 \tag{6}$$
$$\theta_{\texttt{lm_head}} = d \times V = 768 \times 50\,257 = 38\,597\,376 \quad \text{(no bias)} \tag{7}$$
$$\theta_{\text{head}} = 2d + d \cdot V = 38\,598\,912 \tag{8}$$

3.1.4 Total Model Parameters

$$\boxed{\Theta = \theta_{\text{emb}} + L \cdot \theta_{\text{block}} + \theta_{\text{head}} = 39\,383\,808 + 12 \times 7\,087\,872 + 38\,598\,912 = 163\,037\,184 \approx 163\text{M}} \tag{9}$$

$$\mathcal{M}_{\text{param}}^{\text{total}} = \Theta \times b = 163\,037\,184 \times 4 = 652\,148\,736 \text{ bytes} \approx 621.94 \text{ MB} \tag{10}$$

3.2 Per-GPU Parameter Memory (Pipeline Split)

GPipe distributes the $M = 14$ sequential modules across G GPUs according to the balance vector. Let L_i denote the number of *transformer blocks* assigned to GPU i, with the EmbeddingBlock always on GPU 0 and LMHead on GPU $G-1$.

$$\Theta^{(i)} = \mathbf{1}[i = 0]\ \theta_{\text{emb}} + L_i \cdot \theta_{\text{block}} + \mathbf{1}[i = G-1]\ \theta_{\text{head}} \tag{11}$$

$$\mathcal{M}^{(i)}_{\text{param}} = \Theta^{(i)} \times b \tag{12}$$

Example with $G = 2$: Balance $= [7, 7]$. GPU 0 holds modules 0–6 (EmbeddingBlock + 6 TransformerBlocks); GPU 1 holds modules 7–13 (6 TransformerBlocks + LMHead).

$$\Theta^{(0)} = 39\,383\,808 + 6 \times 7\,087\,872 = 81\,911\,040 \qquad \Rightarrow \mathcal{M}^{(0)}_{\text{param}} = 312.47 \text{ MB} \tag{13}$$

$$\Theta^{(1)} = 6 \times 7\,087\,872 + 38\,598\,912 = 81\,126\,144 \qquad \Rightarrow \mathcal{M}^{(1)}_{\text{param}} = 309.47 \text{ MB} \tag{14}$$

Example with $G = 4$: Balance $= [4, 3, 3, 4]$. GPU 0 holds EmbeddingBlock + 3 TransformerBlocks; GPU 3 holds 3 TransformerBlocks + LMHead.

$$\Theta^{(0)} = 39\,383\,808 + 3 \times 7\,087\,872 = 60\,647\,424 \qquad \Rightarrow \mathcal{M}^{(0)}_{\text{param}} = 231.34 \text{ MB} \tag{15}$$

3.3 Gradient Memory $\mathcal{M}_{\text{grad}}$

During the backward pass every parameter accumulates a gradient of the same dtype (`float32`):

$$\mathcal{M}^{(i)}_{\text{grad}} = \Theta^{(i)} \times b = \mathcal{M}^{(i)}_{\text{param}} \tag{16}$$

Gradients are allocated lazily during the first backward pass and persist until `optimizer.zero_grad()` is called.

3.4 Optimizer State Memory $\mathcal{M}_{\text{opt}}$

AdamW maintains two auxiliary states per parameter (first moment m and second moment v), each of the same size as the parameter:

$$\mathcal{M}^{(i)}_{\text{opt}} = 2 \times \Theta^{(i)} \times b = 2 \times \mathcal{M}^{(i)}_{\text{param}} \tag{17}$$

These states are allocated **lazily on the first call** to `optimizer.step()`, i.e., after the first backward pass completes. Before the first step, $\mathcal{M}^{(i)}_{\text{opt}} = 0$.

3.5 Activation Memory $\mathcal{M}_{\text{act}}$

Activation memory is affected by two key features of GPipe:

1. **Micro-batching**: the batch of size B is split into $C = G$ chunks, each of effective size $B_\mu = B/G$.
2. **Activation checkpointing** (`checkpoint='always'`): intermediate activations within each micro-batch partition are *not* saved; instead, only the input tensor to each checkpointed segment is retained, and intermediates are recomputed during the backward pass.

3.5.1 Without Checkpointing (Reference)

For a single transformer block processing one micro-batch, the peak intermediate activations include:

Tensor	Shape	Size (bytes)
Hidden states (input)	$B_\mu \times S \times d$	$B_\mu \cdot S \cdot d \cdot b$
QKV projection	$B_\mu \times S \times 3d$	$3\, B_\mu \cdot S \cdot d \cdot b$
Attention scores	$B_\mu \times H \times S \times S$	$B_\mu \cdot H \cdot S^2 \cdot b$
Attention output	$B_\mu \times S \times d$	$B_\mu \cdot S \cdot d \cdot b$
MLP intermediate	$B_\mu \times S \times d_{\text{ff}}$	$B_\mu \cdot S \cdot d_{\text{ff}} \cdot b$

Per layer, per micro-batch (without checkpointing):

$$\mathcal{A}_{\text{layer}} = B_\mu \cdot S \cdot b\,(d + 3d + d + d_{\text{ff}}) + B_\mu \cdot H \cdot S^2 \cdot b \tag{18}$$

With $B_\mu = 4$, $S = 256$, $d = 768$, $d_{\text{ff}} = 3072$, $H = 12$:

$$\mathcal{A}_{\text{layer}} = 4 \times 256 \times 4 \times (768 + 2304 + 768 + 3072) + 4 \times 12 \times 256^2 \times 4 \approx 31.50 \text{ MB} \tag{19}$$

3.5.2 With Checkpointing (as Used)

With `checkpoint='always'`, GPipe only stores the **input tensor** to each micro-batch partition during the forward pass. All internal activations are recomputed during the backward pass for one segment at a time. The stored activation per layer per micro-batch reduces to:

$$\mathcal{A}_{\text{ckpt}} = B_\mu \cdot S \cdot d \cdot b \tag{20}$$

With $B_\mu = 4$, $S = 256$, $d = 768$, $b = 4$:

$$\mathcal{A}_{\text{ckpt}} = 4 \times 256 \times 768 \times 4 = 3\,145\,728 \text{ bytes} = 3.0 \text{ MB per layer} \tag{21}$$

During the backward pass, one segment's activations are recomputed at a time, and per-layer peak is:

$$\mathcal{A}_{\text{recompute}} = \mathcal{A}_{\text{layer}} \approx 31.50 \text{ MB} \quad \text{(transient, for one layer at a time)} \tag{22}$$

The total activation memory on GPU i during steady-state pipeline execution with C micro-batches is bounded by:

$$\mathcal{M}_{\text{act}}^{(i)} \leq C \cdot L_i \cdot \mathcal{A}_{\text{ckpt}} + \mathcal{A}_{\text{recompute}} \tag{23}$$

The first term accounts for the saved checkpoint inputs across all micro-batches and layers on that GPU; the second term is the transient recomputation cost for the currently active backward segment.

3.6 Miscellaneous Memory $\mathcal{M}_{\text{misc}}$

Additional allocations include:

- **Input tensor**: $B \times S \times 8$ bytes (dtype `int64`) $= 8 \times 256 \times 8 = 16\,384$ bytes ≈ 0.016 MB.
- **Labels tensor**: same size as input, on last GPU.
- **Logits tensor**: $B \times S \times V \times b$ on last GPU $= 8 \times 256 \times 50\,257 \times 4 \approx 391.85$ MB.
- **Causal mask**: $S \times S \times 1$ byte (`bool`) $= 256^2 = 65\,536$ bytes, per layer evaluation.
- **CUDA context and allocator overhead**: typically 200–500 MB (loaded once per GPU, not reflected in `memory_allocated`).
- **Profiler buffers**: memory history recording (`_record_memory_history`) uses additional host and device memory.

4 Peak Memory Formula

The peak memory on GPU i occurs at different training phases. Below we analyze the two critical moments.

4.1 Phase 1: After Model Load, Before Training

Only parameters are allocated:

$$\mathcal{M}^{(i)}_{\text{phase1}} = \mathcal{M}^{(i)}_{\text{param}} \tag{24}$$

4.2 Phase 2: During First Backward Pass (Before First `optimizer.step()`)

At this point, parameters and gradients are live, plus transient activations from checkpointing recomputation:

$$\mathcal{M}^{(i)}_{\text{phase2}} = \mathcal{M}^{(i)}_{\text{param}} + \mathcal{M}^{(i)}_{\text{grad}} + \mathcal{M}^{(i)}_{\text{act}} \tag{25}$$

4.3 Phase 3: Steady State (After First `optimizer.step()`)

The optimizer states now exist, but gradients may be zeroed:

$$\mathcal{M}^{(i)}_{\text{phase3}} = \mathcal{M}^{(i)}_{\text{param}} + \mathcal{M}^{(i)}_{\text{grad}} + \mathcal{M}^{(i)}_{\text{opt}} + \mathcal{M}^{(i)}_{\text{act}} \tag{26}$$

4.4 Overall Peak

The overall peak is achieved during steady-state backward passes (Phase 3):

$$\boxed{\mathcal{M}^{(i)}_{\text{peak}} = 4\,\Theta^{(i)} \cdot b + C \cdot L_i \cdot B_\mu \cdot S \cdot d \cdot b + \mathcal{A}_{\text{recompute}} + \mathcal{M}^{(i)}_{\text{misc}}} \tag{27}$$

where the factor of 4 comes from: parameters ($1\times$) + gradients ($1\times$) + optimizer first moment ($1\times$) + optimizer second moment ($1\times$).

5 Numerical Example

We compute the peak for **GPU 0** using the code's default constants and illustrate with $G = 2$ (the minimum required).

5.1 Step-by-Step Calculation

1. **Balance**: $M = 14, G = 2 \Rightarrow$ balance $= [7, 7]$. GPU 0 holds modules 0–6: EmbeddingBlock + 6 TransformerBlocks, so $L_0 = 6$.

2. **Parameters on GPU 0**:

$$\Theta^{(0)} = \underbrace{39\,383\,808}_{\theta_{\text{emb}}} + 6 \times \underbrace{7\,087\,872}_{\theta_{\text{block}}} = 81\,911\,040 \text{ parameters} \tag{28}$$

3. **Parameter memory**: $\mathcal{M}^{(0)}_{\text{param}} = 81\,911\,040 \times 4 = 327\,644\,160$ bytes $= 312.47$ MB.

4. **Gradient memory**: $\mathcal{M}^{(0)}_{\text{grad}} = 312.47$ MB.

5. **Optimizer state memory**: $\mathcal{M}^{(0)}_{\text{opt}} = 2 \times 312.47 = 624.93$ MB.

6. **Checkpoint activation memory** ($C = 2$ micro-batches, $L_0 = 6$ layers, $B_\mu = 4$):

$$C \cdot L_0 \cdot \mathcal{A}_{\text{ckpt}} = 2 \times 6 \times (4 \times 256 \times 768 \times 4) = 2 \times 6 \times 3\,145\,728 = 37\,748\,736 \text{ bytes} = 36.0 \text{ MB} \tag{29}$$

7. **Recomputation transient**: $\mathcal{A}_{\text{recompute}} \approx 31.50$ MB (one layer at a time).

5.2 Summary Table for GPU 0 ($G = 2$)

Component	**Size (MB)**
Parameters $\mathcal{M}^{(0)}_{\text{param}}$	312.47
Gradients $\mathcal{M}^{(0)}_{\text{grad}}$	312.47
Optimizer states $\mathcal{M}^{(0)}_{\text{opt}}$	624.93
Checkpoint saved activations	36.00
Recompute transient activations	31.50
Input tensor + causal masks	∼0.08
Estimated peak	**∼1 317.45**

Note: The steady-state peak includes all four copies (params + grads + m + v). In practice, the observed peak may be lower if `optimizer.zero_grad()` has freed gradient memory before the optimizer states coexist with new gradients, or if the optimizer states have not yet been allocated (first forward/backward only).

6 Observed Peak and Interpretation

The example trace `peak_alloc_events_GPU0_for example.json` records a peak of:

$$\texttt{max_allocated_mem} = 656\,165\,888 \text{ bytes} \approx 625.77 \text{ MB}$$

This value closely matches $2 \times \mathcal{M}^{(0)}_{\text{param}} \approx 624.93$ MB, suggesting that **the peak occurs during the backward pass when both parameters and gradients are live**, but **before the first** `optimizer.step()` allocates the Adam states (Phase 2). The small excess (∼0.84 MB) corresponds to transient activation tensors and the input data.

This can be expressed as:

$$\boxed{\mathcal{M}^{(0)}_{\text{observed peak}} \approx 2\,\Theta^{(0)} \cdot b + \epsilon = 2 \times 81\,911\,040 \times 4 + \epsilon \approx 625.77 \text{ MB}} \tag{30}$$

where ϵ captures transient activations (∼0.84 MB in this trace).

After the first `optimizer.step()`, the Adam states add another $2 \times \mathcal{M}^{(0)}_{\text{param}} \approx 625$ MB, but subsequent peak measurements via `max_memory_allocated` would reflect that higher plateau.

7 General Formulas (Quick Reference)

Let GPU i hold an embedding indicator $e_i \in \{0, 1\}$, a head indicator $h_i \in \{0, 1\}$, and L_i transformer blocks. Then:

$$\Theta^{(i)} = e_i(Vd + Pd) + L_i(4d^2 + 2d\,d_{\text{ff}} + d_{\text{ff}} + 9d) + h_i(2d + Vd) \tag{31}$$

$$\mathcal{M}^{(i)}_{\text{param}} = \Theta^{(i)} \cdot b \tag{32}$$

$$\mathcal{M}^{(i)}_{\text{grad}} = \Theta^{(i)} \cdot b \tag{33}$$

$$\mathcal{M}^{(i)}_{\text{opt}} = 2\,\Theta^{(i)} \cdot b \quad (\text{AdamW: } m + v) \tag{34}$$

$$\mathcal{M}^{(i)}_{\text{act}} \leq C \cdot L_i \cdot B_\mu \cdot S \cdot d \cdot b + B_\mu \cdot S \cdot b\,(5d + d_{\text{ff}}) + B_\mu \cdot H \cdot S^2 \cdot b \tag{35}$$

$$\boxed{\mathcal{M}^{(i)}_{\text{peak}} = \underbrace{4\,\Theta^{(i)} b}_{\text{params + grads + optimizer}} + \underbrace{C \cdot L_i \cdot B_\mu S d\, b}_{\text{ckpt activations}} + \underbrace{B_\mu S\, b\,(5d + d_{\text{ff}}) + B_\mu H S^2 b}_{\text{recompute transient}} + \mathcal{M}^{(i)}_{\text{misc}}} \tag{36}$$

7.1 Substitution Table

To evaluate for a specific configuration, substitute:

Constant	Code location	Default
B	Line 30: `BATCH_SIZE = 8`	8
G	Line 29: `NUM_GPUS = torch.cuda.device_count()`	runtime
S	Line 32: `SEQ_LEN = 256`	256
V	Line 131: `vocab_size=50257`	50 257
P	Line 132: `n_positions=1024`	1 024
d	Line 133: `n_embd=768`	768
L	Line 134: `n_layer=12`	12
H	Line 135: `n_head=12`	12
d_{ff}	Line 136: `n_inner=3072`	3 072
b	(implicit `float32`)	4

8 Conclusion

By expressing every memory component in terms of the code constants B, G, S, V, P, d, L, H, d_{ff}, and b, we can predict the peak GPU memory usage *before running the script*. The dominant terms are the model parameters and their associated gradients and optimizer states ($4\Theta^{(i)}b$), which scale linearly with model size and inversely with the number of GPUs. Activation memory is significantly reduced by GPipe's built-in checkpointing (`checkpoint='always'`), leaving only $\mathcal{O}(B_\mu \cdot S \cdot d)$ per saved checkpoint. The observed peak of ≈ 625.77 MB on GPU 0 is consistent with $2\Theta^{(0)}b$ (parameters plus gradients during the first backward pass, before optimizer state allocation).